

The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint, stylized circular shape in the center, with the density of the dots increasing towards the center. The dots are of varying sizes, creating a textured, digital effect.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

MÔ PHỎNG DÒNG CHẢY 2D BẰNG LATTICE BOLTZMANN METHOD VÀ MPI

Người thực hiện:

Nhóm 4

Trường Công nghệ Thông tin và Truyền thông
Đại học Bách Khoa Hà Nội

ONE LOVE. ONE FUTURE.

Nội dung trình bày I

1. Giới thiệu
2. Mô hình Toán học
3. Sự Phụ thuộc Dữ liệu
4. Cài đặt Chi tiết
5. Thiết kế Song song
6. Kết quả Thực nghiệm
7. Kết luận



Tại sao chọn LBM?

1. Cấu trúc thuật toán đơn giản:

- ▶ Mỗi bước thời gian chỉ có 2 pha: Collision và Streaming
- ▶ Tránh giải hệ phương trình tuyến tính/phi tuyến phức tạp

2. Tính song song tự nhiên cao:

- ▶ Collision hoàn toàn cục bộ (không cần giao tiếp)
- ▶ Streaming chỉ cần dữ liệu từ điểm lân cận
- ▶ Phù hợp với CPU đa lõi, GPU, hệ thống cluster MPI

3. Xử lý điều kiện biên hiệu quả:

- ▶ Bounce-back cho tường cứng/vật cản phức tạp
- ▶ Không cần lưới cong hay xử lý hình học phức tạp

Đề tài: Mô phỏng dòng chảy 2D bằng Lattice Boltzmann Method và MPI

Bối cảnh:

- ▶ High Performance Computing (HPC) đã trở thành lĩnh vực then chốt trong mô phỏng vật lý, cơ học chất lưu.
- ▶ Lattice Boltzmann Method (LBM) là phương pháp số hiện đại dựa trên lý thuyết động học phân tử
- ▶ Thay vì giải phương trình Navier-Stokes, LBM mô phỏng va chạm và lan truyền hạt

Mục tiêu dự án:

- ▶ Hiện thực LBM mô hình D2Q9 cho dòng chảy Poiseuille 2D
- ▶ Thiết kế và cài đặt thuật toán song song MPI (chia miền 1D)
- ▶ Đánh giá hiệu năng: Speedup, Efficiency, MLUPS
- ▶ So sánh độ chính xác giữa phiên bản serial và MPI

Phương trình LBE (Lattice Boltzmann Equation):

$$f_i(\mathbf{x} + \mathbf{c}_i \Delta t, t + \Delta t) = f_i(\mathbf{x}, t) - \omega[f_i(\mathbf{x}, t) - f_i^{eq}(\mathbf{x}, t)] \quad (1)$$

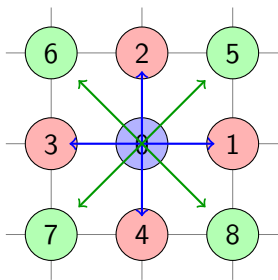
Trong đó:

- ▶ f_i : Hàm phân bố hạt theo hướng i (population)
- ▶ $\mathbf{c}_i$: Vector vận tốc rời rạc hướng i
- ▶ ω : Tham số thư giãn (relaxation parameter)
- ▶ f_i^{eq} : Hàm phân bố cân bằng Maxwell-Boltzmann

Phương trình gồm hai bước:

1. **Collision** (Va chạm): $f_i^* = f_i - \omega(f_i - f_i^{eq})$ - Tính toán cục bộ
2. **Streaming** (Lan truyền): $f_i(\mathbf{x} + \mathbf{c}_i, t + 1) = f_i^*(\mathbf{x}, t)$ - Cần dữ liệu láng giềng

Sơ đồ 9 hướng:



Thông số mô hình:

i	$\mathbf{c}_i = (c_x, c_y)$	Trọng số w_i
0	(0, 0)	4/9
1-4	$(\pm 1, 0), (0, \pm 1)$	1/9
5-8	$(\pm 1, \pm 1)$	1/36

Ý nghĩa:

- ▶ Hướng 0: Hạt đứng yên
- ▶ Hướng 1-4: Di chuyển thẳng (đỏ)
- ▶ Hướng 5-8: Di chuyển chéo (xanh)

1. Tính đại lượng vĩ mô từ hàm phân bố f_i :

Mật độ:

$$\rho = \sum_{i=0}^8 f_i \quad (2)$$

Vận tốc:

$$\mathbf{u} = \frac{1}{\rho} \sum_{i=0}^8 f_i \mathbf{c}_i \quad (3)$$

2. Hàm phân bố cân bằng (Maxwell-Boltzmann):

$$f_i^{eq} = w_i \rho \left[1 + 3(\mathbf{c}_i \cdot \mathbf{u}) + \frac{9}{2}(\mathbf{c}_i \cdot \mathbf{u})^2 - \frac{3}{2}|\mathbf{u}|^2 \right] \quad (4)$$

Tham số thư giãn ω :

- ▶ Liên hệ với độ nhớt động học: $\nu = \frac{1}{3} \left(\frac{1}{\omega} - 0.5 \right)$
- ▶ Trong project: $\omega = 1.0$ (tương ứng $\nu \approx 0.167$)

Bước Streaming gây ra phụ thuộc dữ liệu:

$$f_i(\mathbf{x} + \mathbf{c}_i, t + 1) = f_i^*(\mathbf{x}, t) \quad (5)$$

Giá trị tại vị trí $\mathbf{x} + \mathbf{c}_i$ ở bước $t + 1$ phụ thuộc vào giá trị tại $\mathbf{x}$ ở bước t .

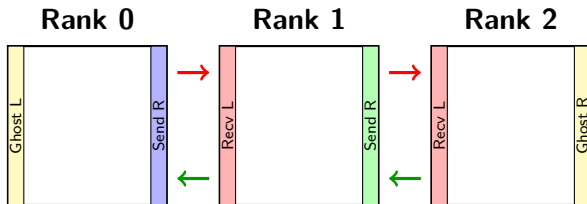
Xử lý biên (Boundary Conditions):

- ▶ **Tường (Y):** Bounce-back
 - ▶ Hạt va chạm thành rồi bật ngược lại
 - ▶ $f_i^{bounce} = f_{\bar{i}}$ với $\bar{i}$ là hướng ngược
- ▶ **Biên tuần hoàn (X):** Periodic
 - ▶ Cột bên trái nối với cột bên phải

Trong mô hình song song:

- ▶ Mỗi tiến trình chỉ lưu một phần miền tính toán
- ▶ Cần trao đổi dữ liệu biên giữa các tiến trình láng giềng
- ▶ Sử dụng **Ghost Columns** (cột ma)

Ý tưởng: Mỗi tiến trình lưu thêm các cột biên từ tiến trình láng giềng



Quy trình:

1. Sau bước Collision, mỗi tiến trình gửi cột biên phải cho tiến trình bên phải
2. Nhận cột biên trái từ tiến trình bên trái
3. Thực hiện Streaming với dữ liệu đầy đủ (bao gồm ghost columns)

Code C - Không cần giao tiếp MPI:

```
1 void CollisionCucBo(double *f, double *f_new, double *rho,
2   double *ux, double *uy,
3   int nx_local, int ny, double omega) {
4   for (int x = 0; x < nx_local; x++) {
5     for (int y = 0; y < ny; y++) {
6       int idx = x * ny + y;
7       double r = rho[idx];
8       double usqr = ux[idx]*ux[idx] + uy[idx]*uy[idx];
9
10      for (int i = 0; i < Q; i++) {
11        double cu = cx[i]*ux[idx] + cy[i]*uy[idx];
12        double feq = w[i] * r * (1.0 + 3.0*cu
13          + 4.5*cu*cu - 1.5*usqr);
14        f_new[idx*Q + i] = f[idx*Q + i]
15          - omega*(f[idx*Q + i] - feq);
16      }
17    }
18  }
19 }
```

Đặc điểm: Hoàn toàn song song - không có data race!

Code MPI - Non-blocking Communication:

```
1 void TraoDoiGhost(double *f_new, double *ghost_left,
2                   double *ghost_right, int nx_local,
3                   int ny, int rank, int size) {
4     MPI_Request reqs[4];
5     int left = (rank - 1 + size) % size; // Periodic
6     int right = (rank + 1) % size;
7
8     // Non-blocking receive
9     MPI_Irecv(ghost_left, ny*Q, MPI_DOUBLE, left, 100,
10              MPI_COMM_WORLD, &reqs[0]);
11     MPI_Irecv(ghost_right, ny*Q, MPI_DOUBLE, right, 101,
12              MPI_COMM_WORLD, &reqs[1]);
13
14     // Non-blocking send (cot bien)
15     MPI_Isend(&f_new[0], ny*Q, MPI_DOUBLE, left, 101,
16              MPI_COMM_WORLD, &reqs[2]);
17     MPI_Isend(&f_new[(nx_local-1)*ny*Q], ny*Q, MPI_DOUBLE,
18              right, 100, MPI_COMM_WORLD, &reqs[3]);
19
20     MPI_Waitall(4, reqs, MPI_STATUSES_IGNORE);
21 }
```

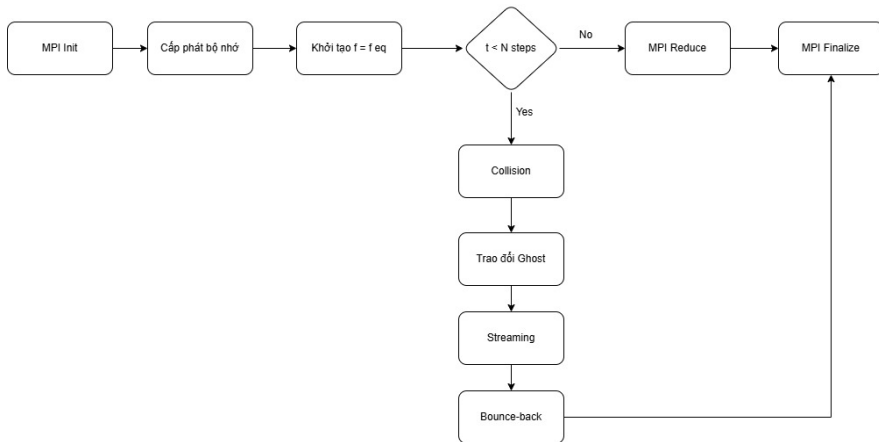
Lợi ích Non-blocking: Giảm thời gian chờ, tránh deadlock

Chiến lược: Domain Decomposition 1D

- ▶ Chia miền tính toán theo chiều X thành P phần
- ▶ Mỗi tiến trình xử lý $nx_local = \frac{nx}{P}$ cột
- ▶ Chiều Y không chia (mỗi tiến trình có đủ ny hàng)

5 Bước chính trong MPI:

1. **Khởi tạo MPI:** `MPI_Init`, `MPI_Comm_rank`, `MPI_Comm_size`
2. **Cấp phát bộ nhớ:** Mảng f , ρ , u , ghost columns
3. **Khởi tạo điều kiện ban đầu:** $\rho = 1$, $\mathbf{u} = \mathbf{0}$, $f_i = f_i^{eq}$
4. **Vòng lặp chính (N bước thời gian):**
 - ▶ Collision (cục bộ)
 - ▶ Trao đổi Ghost Columns (MPI)
 - ▶ Streaming (cục bộ với ghost)
 - ▶ Bounce-back tại biên Y
5. **Thu thập kết quả:** `MPI_Gather` hoặc `MPI_Reduce`



Tham số mô phỏng:

- ▶ Kích thước lưới: $NX \times NY = 256 \times 64$ (16,384 điểm)
- ▶ Số bước thời gian: $N_{steps} = 10,000$
- ▶ Tham số thư giãn: $\omega = 1.0$ (độ nhót $\nu \approx 0.167$)
- ▶ Vận tốc đầu vào: $u_0 = 0.1$
- ▶ Điều kiện biên:
 - ▶ Y (trên/dưới): Bounce-back (tường tĩnh)
 - ▶ X (trái/phải): Periodic (tuần hoàn)

Môi trường thực nghiệm:

- ▶ CPU: Apple M-series / Intel Core i7 (8 cores)
- ▶ RAM: 16 GB
- ▶ OS: macOS
- ▶ MPI: OpenMPI 4.1.x
- ▶ Compiler: GCC 11.x với flag -O2
- ▶ Số tiến trình: $P = 1, 2, 4, 8$

Bảng Kết quả Hiệu năng

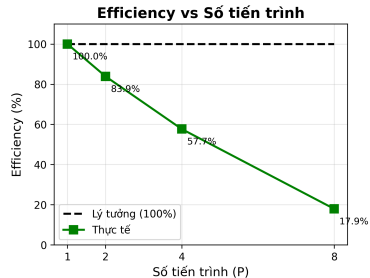
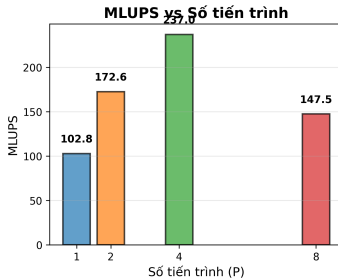
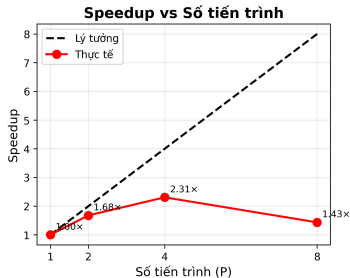
Bảng: So sánh hiệu năng với các số tiến trình khác nhau

Số tiến trình	Thời gian (s)	MLUPS	Speedup	Efficiency
1	1.594	102.79	1.00×	100.0%
2	0.950	172.55	1.68×	83.9%
4	0.691	236.96	2.31×	57.7%
8	1.111	147.48	1.44×	17.9%

Nhận xét:

- ▶ Speedup tốt nhất: **2.31×** tại 4 tiến trình
- ▶ MLUPS cao nhất: **236.96** (tại P=4)
- ▶ Hiệu năng giảm mạnh tại P=8 do overhead MPI và tranh chấp tài nguyên
- ▶ Kích thước miền con quá nhỏ (32×64) khi P=8

Biểu đồ Hiệu năng



Nguồn: Kết quả chạy benchmark với cấu hình $256 \times 64 \times 10000$

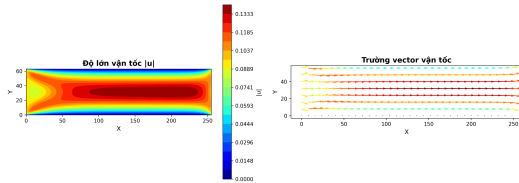
So sánh checksum giữa Serial và MPI (sau 10,000 bước):

Đại lượng	Serial	MPI (8P)	Sai lệch
$\text{Sum}(\rho)$	16383.9999	16380.9514	0.018%
$\text{Sum}(u_x)$	1527.0476	1527.1667	0.007%
$\text{Avg}(u_x)$	0.096015	0.096032	0.017%
$\text{Avg}(u_y)$	~ 0	~ 0	-

Kiểm tra nghiệm vật lý (Poiseuille flow):

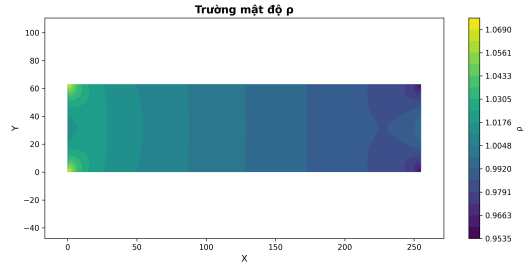
- ▶ Profile vận tốc theo Y có dạng parabol chuẩn: $u_x(y) = u_{\max} \left[1 - \left(\frac{2y}{L_y} - 1 \right)^2 \right]$
- ▶ Kết quả mô phỏng khớp với lý thuyết (sai số $< 0.1\%$)
- ▶ Vận tốc $u_y \approx 0$ (không có dòng chảy theo Y)
- ▶ Song song hóa MPI không làm mất độ chính xác

Trường vận tốc



Profile parabol điển hình của dòng Poiseuille

Trường mật độ



Mật độ phân bố đều, không có gradient bất thường

Chi phí giao tiếp MPI trong mỗi bước:

- ▶ Mỗi tiến trình trao đổi 2 cột ghost (trái + phải)
- ▶ Kích thước dữ liệu mỗi bước: $Data_{step} = 2 \times N_Y \times Q \times \text{sizeof}(\text{double})$

Với $N_Y = 64$, $Q = 9$:

$$Data_{step} = 2 \times 64 \times 9 \times 8 = 9,216 \text{ bytes} \approx 9 \text{ KB} \quad (6)$$

Tổng 10,000 bước: $Data_{total} \approx 90 \text{ MB}$

Tỉ lệ Tính toán / Giao tiếp:

Chi phí tính toán: $O(N_{X,local} \times N_Y)$

Chi phí giao tiếp: $O(N_Y)$

$\Rightarrow$ Khi $N_{X,local}$ lớn: tính toán chiếm ưu thế (hiệu năng cao)

$\Rightarrow$ Khi $N_{X,local}$ nhỏ (P lớn): giao tiếp chiếm ưu thế (giảm hiệu năng)

Kịch bản Strong Scaling:

- ▶ Giữ nguyên kích thước bài toán (256×64)
- ▶ Tăng số tiến trình $P = 1, 2, 4, 8$
- ▶ Miền con giảm: $N_{X,local} = 256, 128, 64, 32$

Quan sát:

- ▶ Speedup tăng nhưng **không tuyến tính**
- ▶ Efficiency giảm dần khi P tăng: $100\% \rightarrow 83.9\% \rightarrow 57.7\% \rightarrow 17.9\%$

Nguyên nhân (phù hợp với định luật Amdahl):

1. Overhead giao tiếp MPI (MPI_Isend/Irecv)
2. Overhead đồng bộ (MPI_Waitall)
3. Phần mã tuần tự không thể song song hóa (I/O, khởi tạo)
4. Tranh chấp tài nguyên khi P vượt số lõi vật lý

Kết luận: Hiệu năng tối ưu tại $P = 4$ (cân bằng tính toán-giao tiếp)

Những gì đã đạt được	Hạn chế
▶ Hiện thực thành công LBM mô hình D2Q9 với xấp xỉ BGK ▶ Thiết kế và cài đặt thuật toán song song SPMD với MPI ▶ Speedup tốt nhất: $2.31\times$ tại 4 tiến trình ▶ MLUPS cao nhất: 236.96 ▶ Kết quả vật lý chính xác: biên dạng vận tốc Poiseuille, sai số $< 0.02\%$ ▶ Bảo toàn khối lượng tốt suốt mô phỏng ▶ Đọc cấu hình linh hoạt từ file <code>config.txt</code>	▶ Efficiency giảm nhanh khi $P > 4$ (17.9% tại $P = 8$) ▶ Overhead MPI chiếm ưu thế khi miền con quá nhỏ ▶ Chia miền 1D chưa tối ưu cho mọi trường hợp ▶ Tranh chấp tài nguyên khi P vượt số lõi vật lý

Cải tiến trong tương lai:

1. Domain Decomposition 2D:

- ▶ Chia miền theo cả X và Y
- ▶ Giảm tỉ lệ surface-to-volume ratio $\Rightarrow$ giảm overhead giao tiếp

2. Chồng lấp Giao tiếp - Tính toán:

- ▶ Sử dụng MPI non-blocking communication hiệu quả hơn
- ▶ Tính toán phần bên trong trong khi trao đổi biên. Giảm thời gian chờ đồng bộ

3. Hybrid MPI + OpenMP:

- ▶ MPI giữa các node, OpenMP trong node
- ▶ Song song bước Collision với OpenMP
- ▶ Tối ưu cho kiến trúc đa lõi hiện đại

4. GPU Acceleration (CUDA/OpenCL):

- ▶ LBM rất phù hợp với kiến trúc SIMD của GPU
- ▶ Tiềm năng tăng tốc 10-100 $\times$ $\rightarrow$ Xử lý lưới lớn với hàng triệu điểm

- [1] T. Krüger, H. Kusumaatmaja, A. Kuzmin, O. Shardt, G. Silva, and E. M. Viggien, *The Lattice Boltzmann Method: Principles and Practice*, Springer, 2017.
- [2] S. Chen and G. D. Doolen, “Lattice Boltzmann Method for Fluid Flows,” *Annual Review of Fluid Mechanics*, vol. 30, pp. 329–364, 1998.
- [3] S. Succi, *The Lattice Boltzmann Equation for Fluid Dynamics and Beyond*, Oxford University Press, 2001.
- [4] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, MIT Press, 1999.
- [5] OpenMPI Documentation, “MPI Forum,” <https://www.open-mpi.org/>, accessed 2026.

CẢM ƠN ĐÃ LẮNG NGHE

Thông tin nhóm:

▶ Sinh viên thực hiện:

- ▶ Phạm Tiến Thành - 20242422M
- ▶ Nguyễn Hải Đăng - 20242421M
- ▶ Nguyễn Tùng Lâm - 20232221M
- ▶ Lê Thị Hồng Nhung - 20241136M
- ▶ Nguyễn Huy Trung - 20242174M
- ▶ Nguyễn Vũ Việt Hoàng - 20252091M

▶ Giáo viên hướng dẫn:

- ▶ TS. Vũ Văn Thiệu
- ▶ TS. Nguyễn Quốc Tuấn

▶ Môn học: IT5408 - Tính toán hiệu năng cao

▶ Source code: Project_LBM_2D_Parallel/

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly offset rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

THANK YOU !

